45. Numerics and Tensors in Go (gonum)

45. Numerics and Tensors in Go (gonum)

Phase: Phase 7 — AI engineering in Go
Estimated time: 2 days
Slug: `45-numerics-gonum`

Learning objectives

- Use float64 carefully
- Perform vector ops with gonum-style APIs
- Know floating-point pitfalls
- Benchmark numeric kernels

Prerequisites

Complete prior numbered lessons. Type every example yourself.

Floating point

Floats are approximations. Never compare with == without tolerance. Summation order changes results; catastrophic cancellation is real.

gonum

The gonum organization provides matrices, stats, graph, optimize packages — Go's scientific backbone.

```
// Dot product
func Dot(a, b []float64) float64 {
    var s float64
    for i := range a { s += a[i] * b[i] }
    return s
}
```

AI relevance

Even when models run elsewhere, Go services still normalize features, compute similarities, and batch tensors as bytes.

Common mistakes (especially from Python instincts)

- Equality checks on floats
- Ignoring NaN propagation
- Using float32 without need then losing precision

Exercises

Work in `exercises/45-numerics-gonum/`. Read that folder's `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.

- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/45-numerics-gonum/` until you have an honest failing attempt.

Self-check

1. Why epsilon comparisons?
2. What is gonum used for?
3. Dot product complexity?

Further reading

- https://www.gonum.org/